

Employee Database Management System

DDL Commands and Data Integrity Constraints

Prepared By	Dhanalakshmi A
Course	Data Analytics
Module	MySQL DDL Commands & Constraints

Project Objective

This project involves designing an Employee Database for an organization that needs to track its employees, the departments they belong to, and the locations they work from. The goal is to build the database using MySQL DDL commands and to apply the right constraints so the data stays accurate, consistent, and free of duplicate or missing values.

DDL Commands Covered

- CREATE – to build the database and its tables
- ALTER – to add, modify, drop, or rename columns
- RENAME – to rename tables
- TRUNCATE – to clear all records from a table
- DROP – to remove tables and the database

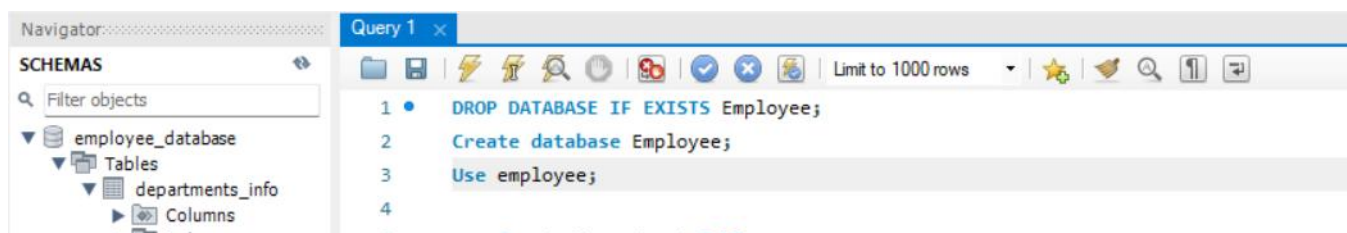
Creating the Database and Tables

The project uses three related tables. Departments and Location each hold their own primary key, and the Employees table connects to both of them through foreign keys, so every employee record can be traced back to its department and its work location.

Step 1: Create the Database

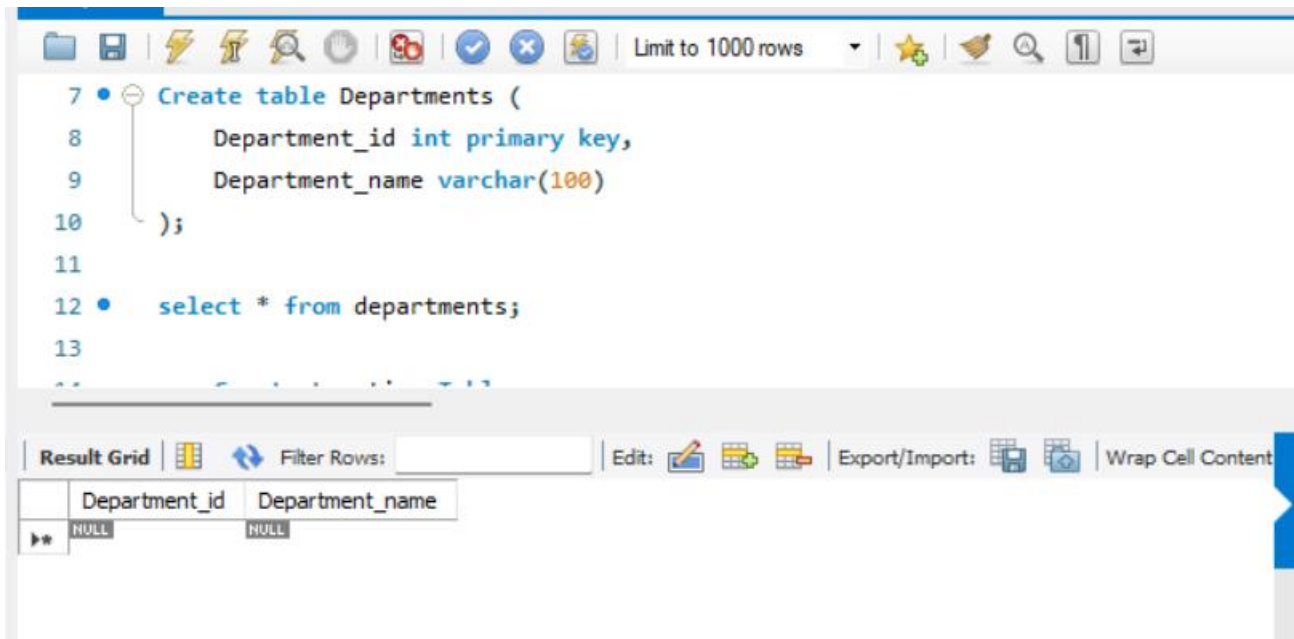
Create database Employee_db;

Use employee_db;



Step 2: Create the Departments Table

```
Create table Departments(  
    department_id int auto_increment primary key,  
    department_name varchar(50)  
);
```

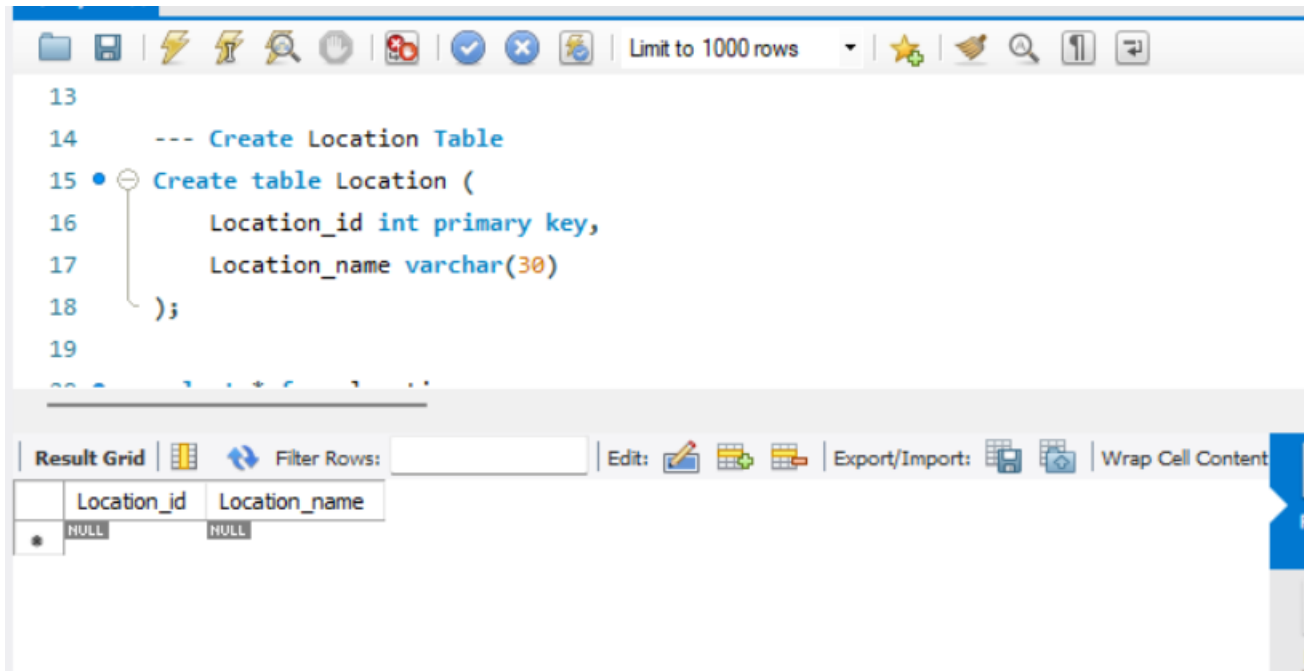


Step 3: Create the Location Table

```

Create table Location(
    location_id int auto_increment primary key,
    location_name varchar(100)
);

```



Step 4: Create the Employees Table

The Employees table is linked to both Departments and Location using foreign keys, which is what establishes the relationship between the three tables.

```

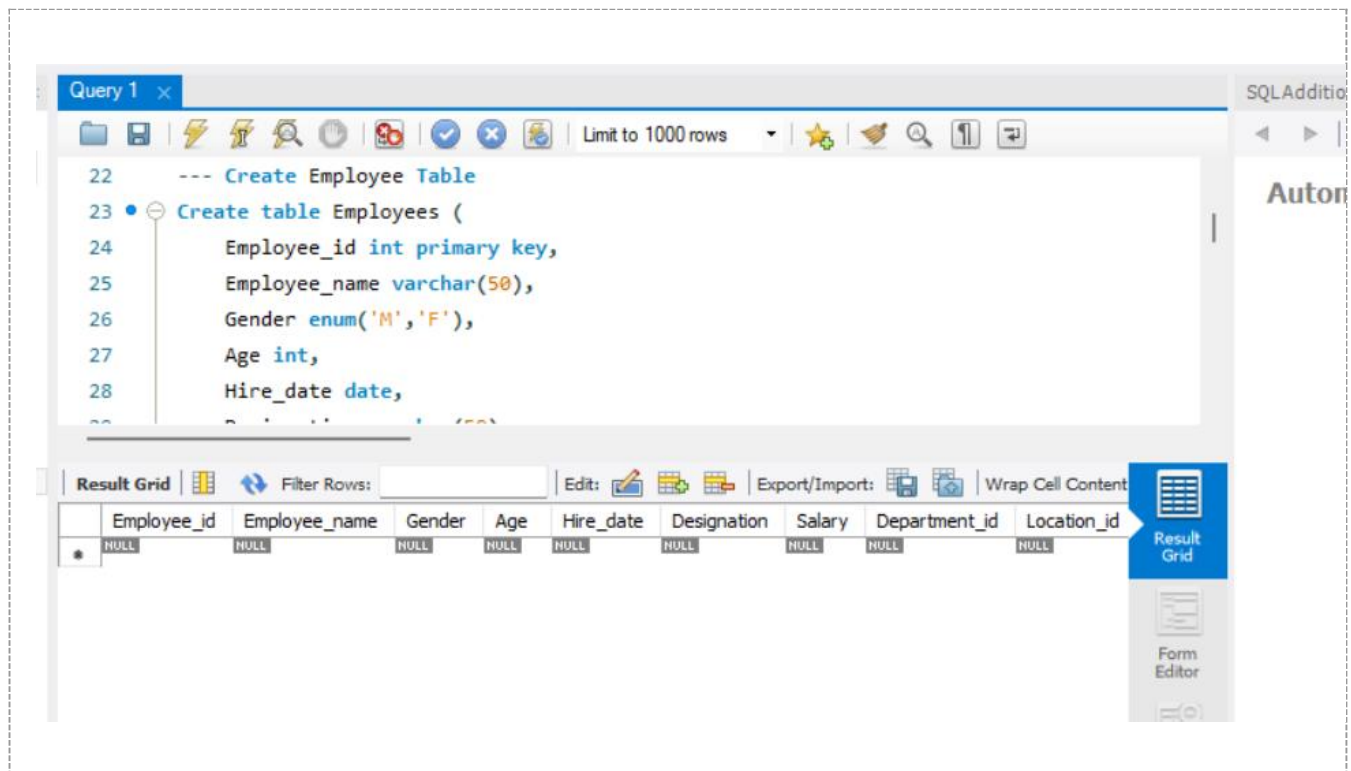
Create table Employees(
    employee_id int auto_increment primary key,

```

```

employee_name varchar(100),
gender enum('M','F'),
age int,
hire_date date,
designation varchar(100),
salary decimal(10,2),
department_id int,
location_id int,
foreign key (department_id) references Departments(department_id),
foreign key (location_id) references Location(location_id)
);

```



Altering the Employees Table

Add the email column

Alter table Employees
Add column email varchar(100);

```

40
41 --- Add new column "email"
42 Alter table Employees
43 Add column Email varchar(100);
44

```

Widen the designation column

Alter table Employees
Modify designation varchar(255);

```
44
45    --- Modify "designation" to hold a wider range of values
46 •   Alter table Employees
47     Modify column Designation varchar(150);
48
```

Drop the age column

Alter table Employees
Drop column age;

```
49    --- Drop the "age" column
50 •   Alter table Employees
51     Drop column Age;
52
```

Rename hire_date to date_of_joining

Alter table Employees
Rename column hire_date to date_of_joining;

```
-- Rename "hire_date" to "date_of_joining"
•   Alter table Employees
    Rename column Hire_date to Date_of_joining;
```

Renaming the Tables

Rename table Departments to Departments_Info;

Rename table Location to Locations;

```
-- Table Renaming (RENAME)

•   Rename table Departments to Departments_Info;
•   Rename table Location to Locations;
```

Truncating a Table

Truncate removes every row from a table instantly while keeping the table structure intact, which is faster than deleting rows one at a time.

Truncate table Employees;

Dropping Tables and the Database

Drop table Employees;

Drop database Employee_db;

Part F – Rebuilding the Database with Constraints

Once the DDL commands were tested, the database was rebuilt from scratch so that every table would carry proper constraints from the start. This step focuses on data integrity rather than just structure.

Drop database if exists Employee_db;

Create database Employee_db;

Use employee_db;

Departments Table

department_id is set as the primary key so every department has a distinct number, and department_name is marked NOT NULL and UNIQUE so it can never be left blank or repeated.

```
Create table Departments(  
    department_id int primary key auto_increment,  
    department_name varchar(50) not null unique  
);
```

Location Table

location_id uses AUTO_INCREMENT so MySQL generates a new sequential number for every location automatically, and location_name is NOT NULL and UNIQUE to stop blank or duplicate entries.

```
Create table Location(  
    location_id int primary key auto_increment,  
    location_name varchar(100) not null unique  
);
```

Employees Table

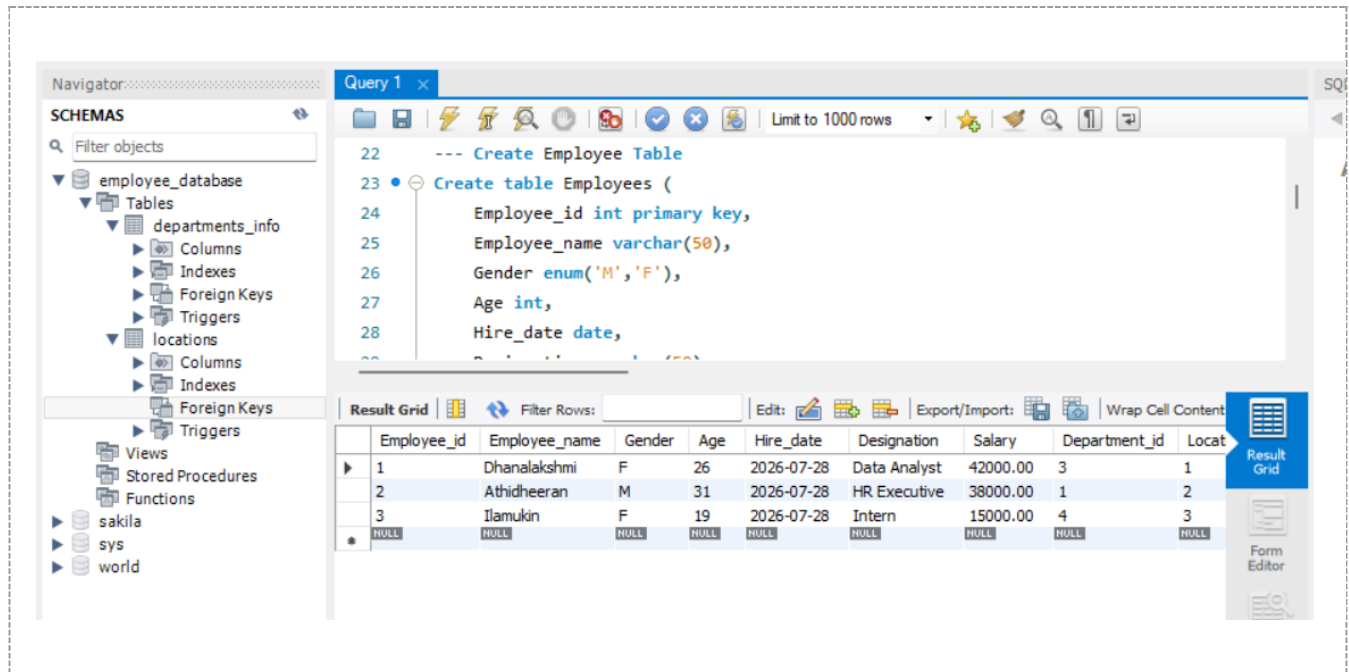
- employee_id – primary key with auto_increment, so every employee gets a unique ID
- employee_name – NOT NULL, so a record cannot be saved without a name
- gender – restricted with ENUM('M','F') so only those two values are accepted
- age – CHECK (age >= 18) blocks any employee below 18 from being added
- hire_date – DEFAULT (current_date) fills today's date automatically if it isn't provided
- department_id and location_id – FOREIGN KEYS linking back to Departments and Location

```
Create table Employees(  
    employee_id int primary key auto_increment,  
    employee_name varchar(100) not null,  
    gender enum('M','F'),  
    age int check (age >= 18),  
    hire_date date default (current_date),  
    designation varchar(255),  
    salary decimal(10,2),
```

```

department_id int,
location_id int,
email varchar(100) unique,
foreign key (department_id) references Departments(department_id),
foreign key (location_id) references Location(location_id)
);

```



Testing the Constraints with Sample Data

A few sample rows were inserted into each table to confirm the constraints behave as expected.

Insert into Departments(department_name)
 Values('HR'),('Finance'),('IT'),('Sales');

Insert into Location(location_name)
 Values('Madurai'),('Chennai'),('Coimbatore'),('Bangalore');

Insert into Employees(employee_name, gender, age, designation, salary, department_id, location_id, email)
 Values
 ('Dhanalakshmi','F',26,'Data Analyst',42000,3,1,'priya.ramesh@example.com'),
 ('Athidheeran','M',31,'HR Executive',38000,1,2,'vignesh.kumar@example.com'),
 ('Ilamukin','M',19,'Intern',15000,4,3,'anitha.selvam@example.com');

Conclusion

- The Employee database was built step by step using CREATE, ALTER, RENAME, TRUNCATE, and DROP.
- Departments, Location, and Employees were designed as three connected tables.
- PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints were applied to keep the data reliable.
- Invalid entries, such as an age under 18 or a gender outside M/F, are rejected automatically by the constraints.
- Sample records were inserted and checked to confirm every constraint works as intended.

Thank You!